

Comments on Black-Scholes PMM Notebook

PC

General Comments

These notebooks contain a significant amount of well-written information for this new application of PMMs and competing methods. The extrapolation of the admittedly limited information in our published paper to this code is impressive, even disregarding the short time frame in which this was accomplished. I personally am very interested in pushing this as far as possible. Despite the PMM performing admirably, I think there are multiple algorithmic improvements to the PMM form that could allow the PMM to completely dominate the existing methods. I will try to outline these ideas in this document, but I would also be happy to discuss them in more detail if desired.

Somewhat selfishly, I also want to advertise the library I have written specifically for PMMs, which is available at <https://parametric-matrix-models.github.io/pyPMM/>. It (and especially the documentation) is still a work in progress, but is fully functional and supports both CPU and GPU training and inference. Its main benefit over `torch` in the context of these notebooks is that it uses `jax` for compilation and automatic differentiation, which we have found to be both faster and more versatile (especially in regards to complex numbers, both those aren't relevant here). Using this library would allow for modularity of the PMM model, not unlike the modularity of layers in artificial neural networks, but more importantly would enable the use of more advanced and accurate ODE/PDE solvers. I'd be happy to implement the PMM for the 1D Black-Scholes equation using this library and advanced ODE/PDE solvers to serve both as a reference implementation for the library and as an example of many of my comments in this document, if desired.

Projection of the Black-Scholes Equation into the Latent Space

In the notebook, the Black-Scholes equation is projected into the latent space as follows:

$$\frac{\partial C}{\partial t} = \left[-\frac{1}{2}\sigma^2 S^2 \frac{\partial^2}{\partial S^2} - rS \frac{\partial}{\partial S} + rI \right] C \xrightarrow{\text{POD}} \frac{\partial c}{\partial t} = \left[-\frac{1}{2}\sigma^2 s^2 A - rsB + rI \right] c,$$

where $c = P^T C$ and $s = P^T S$ are the latent space representations of C and S , respectively. This is not exactly correct for a few subtle reasons. However, the issues are corrected via the learned parameters themselves during training, so the immediate results are not affected, but these issues may be the reason for issues with rescaling operators for different values of σ and may lead to poorer performance in general. To illustrate the issues, I will write out the projection in pedantic detail. Starting by writing the discretized Black-Scholes equation in Einstein summation notation,

$$\frac{\partial C_i}{\partial t} = \left[-\frac{1}{2}\sigma^2 S_i^2 (\mathcal{D}^2)_{ij} - rS_i (\mathcal{D})_{ij} + rI_{ij} \right] C_j,$$

where $\mathcal{D}$ is the discretized first derivative operator with respect to S , and $\mathcal{D}^2$ is the discretized second derivative operator with respect to S . Now the most important difference between this and the projection in the notebook is that **$\mathbf{S}$ is not dependent on the state vector $\mathbf{C}$** , or time t , but rather is a fixed vector of the discretized S values. As such, there is no need to project S into the latent space. In a separate note that Morten should have provided, I show how one would project equations involving similar elementwise multiplications, which are nonlinear operations and require special considerations. However, since S is fixed, we can and should simply absorb the S values into the derivative operators themselves:

$$\frac{\partial C_i}{\partial t} = \left[-\frac{1}{2}\sigma^2 (\tilde{\mathcal{D}}^2)_{ij} - r(\tilde{\mathcal{D}})_{ij} + rI_{ij} \right] C_j,$$

where $\tilde{\mathcal{D}}_{ij} = S_i \mathcal{D}_{ij}$ and $\tilde{\mathcal{D}}^2_{ij} = S_i^2 \mathcal{D}_{ij}^2$ are the modified derivative operators that include the S values. Now we can project this equation into the latent space by applying P^T to the state vector and $P^T(\cdot)P$ to the operators:

$$\begin{aligned} \frac{\partial (P^T C)_i}{\partial t} &= \left[-\frac{1}{2} \sigma^2 \left(P^T \tilde{\mathcal{D}}^2 P \right)_{ij} - r \left(P^T \tilde{\mathcal{D}} P \right)_{ij} + r (P^T I P)_{ij} \right] (P^T C)_j \\ \implies \frac{\partial c_i}{\partial t} &= \left[-\frac{1}{2} \sigma^2 A_{ij} - r B_{ij} + r E_{ij} \right] c_j, \end{aligned}$$

where now A, B, E are the projected operators to be learned in the latent space. Herein lies a second subtlety: while it is reasonable to set $E = I$ in the latent space, since P is orthonormal, in order for the PMM to take full advantage of the “floating basis” idea, it may be beneficial to learn E as a general symmetric matrix. When data is limited however, reducing the number of learned parameters by setting $E = I$ may be beneficial.

Finally, the operators A and B are not necessarily symmetric, due to both how the derivatives may be discretized and how the S values are absorbed into the operators. Enforcing symmetry or Hermiticity of the learned operators is only done when the POD transformation suggests that the latent space operators should have these symmetries. Training of the PMM is not contingent on these operators having these symmetries.

Solving Black-Scholes with Enforced Boundary Conditions via Time-Reversal

When generating the training data in the 1D case using a numerical solver (as opposed to the closed form used in the notebook), or when performing the forward pass in any of the PMMs, it is important to enforce the boundary conditions of the Black-Scholes equation,

$$\begin{aligned} C(0, t) &= 0 \text{ for all } t, \\ C(S, T) &= \max(S - K, 0). \end{aligned}$$

The first condition is easy to enforce in the original full space, since it is simply a matter of setting the derivative of the $S = 0$ element to zero and ensuring that $C(0, t = 0) = 0$, i.e. the constraint is satisfied in the initial condition. A common way to enforce the second condition is to use time-reversal, i.e. solve the equation backwards in time from $t = T$ to $t = 0$. This can be done by simply making the substitution $\tau = T - t$, which transforms the Black-Scholes equation into

$$\begin{aligned} \frac{\partial C}{\partial \tau} &= \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 C}{\partial S^2} + r S \frac{\partial C}{\partial S} - r C, \\ &\text{subject to} \\ C(0, \tau) &= 0 \text{ for all } \tau, \\ C(S, 0) &= \max(S - K, 0). \end{aligned}$$

This gives a simple initial value problem with an easily enforced time-independent boundary condition. Let $c = P^T C$ be the latent space representation of C , and thus $C \approx P c$. Switching to Einstein summation notation where i indexes the discretized S values, so $C_i \approx P_{ij} c_j$. Let $\hat{C}_i = P_{ij} c_j$ be the predicted (reconstructed) value of C_i from the latent space, then the boundary condition at $S = 0$ becomes $\hat{C}_0 = P_{0i} c_i = 0$ for the boundary condition at $S = 0$. This will almost certainly be true automatically if the training data was generated with the boundary condition enforced, which can be checked by asserting that the first row of P is all zeros. In the notebook for the 1D case, the maximum absolute value of any element in the first row of P is $\sim 10^{-12}$, which is pretty much zero. In the time-reversed transformed PMM, the second boundary condition is automatically (approximately) satisfied by the initial condition projected from the full space. This should address issues with the PMM’s error increasing near $S = K, t = T$. This same time-reversal approach can be applied to the TNN and PINN as well, but will not guarantee (approximate) adherence in that case.

I know next to nothing about financial mathematics, but solving the Black-Scholes equation in a *non*-time-reversed manner seems strange to me. In practice, surely one wouldn’t know the current price $C(S, 0)$, but would know the price at maturity. This doesn’t affect the comparisons between the methods, but it had me confused about the context of the problem for a while.

Reducing the Number of Learned Operators in the Latent Space

The latent representations of $S \frac{\partial}{\partial S}$ and $S^2 \frac{\partial^2}{\partial S^2}$ need not be two separate learned operators, since

$$\mathcal{D} \equiv S \frac{\partial}{\partial S} \implies \mathcal{D}^2 = \mathcal{D}\mathcal{D} = S \frac{\partial}{\partial S} S \frac{\partial}{\partial S} = S^2 \frac{\partial^2}{\partial S^2} + S \frac{\partial}{\partial S}.$$

Therefore, one can transform the original Black-Scholes equation into the form

$$\begin{aligned} \frac{\partial C}{\partial t} &= -\frac{1}{2}\sigma^2 \left[S^2 \frac{\partial^2 C}{\partial S^2} + S \frac{\partial C}{\partial S} \right] - rS \frac{\partial C}{\partial S} + \frac{1}{2}\sigma^2 S \frac{\partial C}{\partial S} + rC \\ &= -\frac{1}{2}\sigma^2 \left[S^2 \frac{\partial^2 C}{\partial S^2} + S \frac{\partial C}{\partial S} \right] - \left(r - \frac{1}{2}\sigma^2 \right) S \frac{\partial C}{\partial S} + rC \\ &= -\frac{1}{2}\sigma^2 \mathcal{D}^2 C - \left(r - \frac{1}{2}\sigma^2 \right) \mathcal{D} C + rC \end{aligned}$$

where $\mathcal{D} = S \frac{\partial}{\partial S}$ and $\mathcal{D}^2 = \mathcal{D}\mathcal{D}$. This means that you only need to learn two operators in the latent space, $\mathcal{D}$ and the identity operator—or just one if the identity operator stays fixed in the latent space. This is especially useful for the higher-dimensional case, where you will only need a single operator for each dimension, rather than the number of dimensions squared.

Other Linear Projectors

It may be worth experimenting with other linear projectors than the POD (SVD/PCA). Specifically, Non-negative Matrix Factorization (NMF) may be useful here, since (I assume) the prices are always non-negative.

Comments on Selected Summary Items

- 2 *The PMM can only predict using the same dt as during training (at least I have not found a way to predict using different dt). This may cause such a PMM model to be less generalizable compared to methods such as traditional neural networks and PINNs.*
 - I have several ODE/PDE PMM models that can accept arbitrary time steps during prediction, and in fact it seems like the code in the notebook should be able to do this as well. Before calling `pmm.predict_full`, you should be able to, for instance, set `pmm.dt = dt/10` and then call `pmm.predict_full` with $N_t = 10 \times N_t$ to get predictions at a finer time resolution.
 - At the expense (and benefit) of ditching `torch` for `jax`, you might take a look at the `diffjax` library, which has autodiff-able ODE/PDE solvers beyond the standard Euler method.
- 5 *The PMM model in this file can only predict using the same set of financial parameters as was used during training (the ones used for the Monte-Carlo simulation). It would be interesting to see if one could define the model to generalize to different parameters as well. One avenue of research would be to include the parameters when projecting the data to the lower-dimensional space for training. After the training we project back to the full space, and scale the parametrized matrices to match a new set of financial parameters.*
 - Interpolation/extrapolation in model parameters is exactly what PMMs were designed for. The idea would be to ensure your training data includes a wide range of parameter values and include those parameters as part of the input to the PMM. Since you know the exact dependence of the model (and thus the PMM) on the parameters, you can separate these parameters from the state variables and use them to form the operators for the PMM. In summary, your training data is currently in the form of samples of $(\vec{C}(t), \vec{C}(t+dt))$ for a fixed set of parameters $\vec{\sigma}$; simply change the input data to concatenate the parameters, i.e. $([\vec{C}(t), \vec{\sigma}], \vec{C}(t+dt))$, and during the forward pass extract these parameters to form the operators for the PMM. I have done this exact procedure for a PMM for the time evolution of BECs in 2D. Since the model is quadratic in the parameters, there is no need to project the parameters, only the state vectors.

- In the discussion of training on $\sigma_i = 0.2$ and then rescaling the second derivative matrices by 2 in order to predict for $\sigma_i = 0.4$: I think there is a small mistake here, the matrices should be scaled by 4 instead of 2, since the second derivative operator is scaled by $\sigma_i \sigma_j$. This may have been what prevented these experiments from working.

9 *It is also needed to do a more careful analysis and discussion of why the PMM performs better than the PINN model and traditional model, especially when increasing the number of dimensions.*

- This likely stems from the model-order-reduction nature of the PMM. As the dimensionality of the problem increases, the number of effective degrees of freedom (think principal components) doesn't necessarily increase at the same rate, or at all. In essence, the subspace projection (SVD) is doing a lot of heavy lifting here. If you applied PCA or KPCA to the data before training the TNN or PINN, you might find that they perform similarly well to the PMM, so long as there is sufficient training data.

Disclaimer

Please see XKCD 3126: <https://xkcd.com/3126/>